

Implementação de uma Árvore Geradora Mínima Paralela usando Memória Distribuída

Igor Roberto Michalski de Souza RGM 47539

Ano: 4 Turma: U

1 de julho de 2026

1 Sumário do problema

Dado um grafo não direcionado, conexo e ponderado $G = (V, E)$, uma Árvore Geradora Mínima (*Minimum Spanning Tree*, MST) é um subconjunto $T \subseteq E$ que forma uma árvore conectando todos os vértices de V e cujo peso total $\sum_{e \in T} w(e)$ é o menor possível [1]. O problema é clássico em teoria dos grafos e possui aplicações diretas em projeto de redes de comunicação e redes de computadores, além de aplicações indiretas em visão computacional e análise de *clusters*.

Este trabalho implementa o algoritmo proposto pelo professor da disciplina, que constrói a MST de forma iterativa a partir de componentes. Inicialmente cada vértice forma um componente isolado. A cada rodada, todo componente seleciona a aresta de menor peso que o conecta a um componente distinto; essas arestas são adicionadas à floresta e os componentes ligados por elas são fundidos. O processo se repete até restar um único componente contendo todos os vértices, momento em que a MST está completa.

O objetivo central é implementar o algoritmo em linguagem C utilizando a API MPI, executando sobre o cluster do laboratório de Ciência da Computação da UEMS em memória distribuída. São desenvolvidas duas versões: uma sequencial e uma paralela (executada em 4, 8, 12 e 16 processos). Exige-se que ao menos uma das configurações paralelas alcance eficiência $\geq 70\%$. Ao final, o trabalho apresenta os gráficos de tempo, speedup, eficiência e a análise pela Lei de Amdahl que caracterizam o ganho obtido com o paralelismo.

2 Algoritmos e estruturas de dados

Descrevem-se a seguir o fluxo do algoritmo, o tipo abstrato de dados que sustenta o controle de componentes e as principais funções.

2.1 Versão sequencial

A entrada é um arquivo binário com a sequência de arestas do grafo, cada uma ocupando 16 bytes: dois inteiros de 4 bytes para os vértices de origem e destino e um double de 8 bytes para o peso. O arquivo não possui cabeçalho; por isso, antes do processamento, o programa o percorre uma vez para determinar o número de vértices e dimensionar as estruturas de trabalho. Em seguida, cada vértice é inicializado como uma componente isolada.

O algoritmo opera por rodadas, repetindo o procedimento da Parte I da especificação até restar uma única componente. Cada rodada tem duas fases:

- **1:** o vetor de candidatas é reiniciado e o arquivo é lido em blocos de 16 MB. Para cada aresta, identificam-se as componentes (raízes) das duas pontas; se forem distintas, a aresta

disputa o posto de aresta mais barata de cada uma dessas componentes, registrada em `menor[raiz]`.

- **2:** percorre-se o vetor de candidatas; para cada componente que encontrou uma aresta, se as duas pontas ainda estiverem em componentes distintas, as componentes são unidas e a aresta é incorporada à MST.

Entre as duas fases há uma verificação de parada: se nenhuma componente encontrou aresta durante a varredura, o grafo já é uma única componente e o laço termina. Como cada rodada funde toda componente ainda conectável a pelo menos uma outra, o número de componentes cai ao menos pela metade por rodada e o total de rodadas cresce com o logaritmo do número de vértices, ainda que o arquivo seja relido por completo a cada rodada. Ao final, mede-se o tempo de execução, obtém-se o peso total da árvore e as arestas da MST são gravadas em arquivo.

2.2 Versão paralela

A versão paralela segue o mesmo algoritmo por componentes, mas as arestas são divididas entre os processos: cada processo guarda apenas uma fatia das arestas. O union-find (componente e altura) é replicado, ou seja, todo processo mantém a estrutura completa e aplica as mesmas uniões. O que se divide é a busca pela aresta mais barata de cada componente; a decisão de fusão é tomada de forma idêntica por todos a partir de um resultado reduzido globalmente. Com isso, o passo de união não troca mensagens: a consistência vem de todos partirem do mesmo dado e do desempate determinístico.

Cada rodada tem três momentos. Primeiro, cada processo calcula o seu `menor_local`: para cada componente vivo, a aresta mais barata que a sua fatia enxerga (a função `atualizar_menor()` aplica o desempate), usando o sentinela `DBL_MAX` para marcar “Nenhuma” nos componentes sem aresta de saída naquela fatia. Em seguida, uma operação coletiva `MPI_Allreduce()` com o operador comutativo `op_menor_aresta()` combina os vetores `menor_local` de todos os processos, posição a posição, escolhendo a mais barata com o mesmo desempate; o resultado é o vetor `menor_global[]`, com a aresta mais barata real de cada componente. Por fim, como todos têm o mesmo `menor_global[]`, cada processo aplica localmente a mesma sequência de uniões, com a segunda verificação de mesma componente, e apenas o processo 0 grava as arestas escolhidas na árvore. Os vetores `menor_local[]` e `menor_global[]` são dimensionados pelo número de componentes vivos, e não por V , de modo que o vetor reduzido encolhe à medida que os componentes se fundem.

Na rodada 1, cada vértice é seu próprio componente, então a aresta mais barata de cada componente pode ser calculada usando o próprio identificador do vértice como índice, sem consultar componente. Isso é aproveitado: enquanto as arestas ainda estão sendo distribuídas, a função `dobrar_rodada1()` já acumula o `menor_local[]` da rodada 1 sobre cada fatia e, na mesma passada, descobre o maior identificador de vértice. Esse ponto é necessário porque o número de vértices só é conhecido após essa passada, por um `Allreduce` do maior identificador, então componente e altura ainda não estão alocados. Uma sinalização de primeira rodada faz a primeira iteração do laço principal pular a varredura e reaproveitar o `menor_local` já preenchido durante a distribuição. Na prática, a Fase 1 da rodada 1 ocorre sobreposta à transferência dos dados.

Da rodada 2 em diante, duas otimizações mantêm cada rodada barata. Antes da varredura, uma passada $O(|V|)$ achata componente de forma explícita, fazendo todo vértice apontar direto para a sua raiz atual. Com isso, durante a varredura a consulta da raiz de um vértice deixa de percorrer a cadeia de pais e passa a ser uma leitura direta em componente, um acesso $O(1)$, o que ainda favorece a localidade de cache ao reduzir a poda das arestas a acessos simples ao vetor. Além disso, a fatia de arestas é podada: as arestas internas, com as duas pontas no mesmo componente, são removidas em definitivo por meio de um cursor de escrita, de modo que a fatia encolhe a cada rodada e as rodadas seguintes varrem menos arestas. O laço termina quando uma rodada não realiza nenhuma união, pois então o grafo já é um único componente.

2.3 Tipo abstrato de dados: union-find

O controle de quais vértices já estão conectados é feito por um union-find (conjuntos disjuntos), representado por dois vetores de inteiros de tamanho $|V|$. O vetor componente guarda, para cada vértice, o seu pai imediato; a componente é identificada pela sua raiz, o vértice que aponta para si mesmo. O vetor altura guarda a estimativa da altura por raiz.

Duas operações sustentam a estrutura. `achar componente()` sobe a cadeia de pais até a raiz aplicando *path halving*, cada nó visitado passa a apontar para o avô, o que mantém as árvores rasas e barateia as buscas seguintes. `juntar()` une duas componentes fazendo a raiz da árvore mais baixa apontar para a da mais alta. A altura só aumenta quando ambas têm altura igual. Essa combinação mantém a busca por componentes em tempo praticamente constante, bastando comparar as raízes [2]. Vale notar que, com *path halving*, altura deixa de ser a altura exata e passa a ser apenas uma estimativa, ainda suficiente para decidir a direção das uniões.

2.4 Principais funções

A Tabela 1 e Tabela 2 resumem o papel de cada uma das principais funções para cada uma das implementações.

Função	Papel
<code>achar_componente</code>	Operação <i>Find</i> do union-find, com <i>path halving</i> .
<code>juntar</code>	Operação <i>Union</i> do union-find, por <i>rank</i> (altura).
<code>for_preferido_sobre</code>	Decide se uma aresta é mais barata que a candidata atual de uma componente, pela regra $\text{peso} \rightarrow \text{origem} \rightarrow \text{destino}$.

Tabela 1: Principais funções da versão sequencial.

Função	Papel
<code>dobrar_rodada1</code>	Acumula as menores arestas da rodada 1 enquanto a fatia é distribuída e, na mesma passada, descobre o maior identificador de vértice.
<code>garantir_capacidade</code>	Cresce dinamicamente o vetor <code>menor_local</code> (dobrando a capacidade) à medida que novos componentes aparecem, já que o número de vértices é desconhecido na rodada 1.
<code>op_menor_aresta</code>	Operador comutativo do Allreduce que combina os <code>menor_local</code> , escolhendo a aresta mais barata de cada componente pelo desempate $\text{peso}, \text{origem}, \text{destino}$.

Tabela 2: Principais funções específicas da versão paralela.

3 Decisões de implementação não especificadas

A especificação descreve o algoritmo em alto nível, tratando a entrada como uma matriz de pesos $W(u, v)$ e deixando a regra de desempate abstrata, enquanto a entrada real é um arquivo binário de arestas sem metadados.

O grafo é tratado como uma lista de arestas, com uma struct de 16 bytes idêntica ao layout do arquivo, e não como a matriz $W(u, v)$ do pseudocódigo, que seria inviável para a ordem de grandeza do problema por exigir espaço V^2 . Como o arquivo não traz a contagem de vértices, ela é obtida em uma passada inicial que busca o maior identificador, e o número de vértices é definido como esse maior identificador mais um, o que dimensiona as estruturas. Por não caber

em memória, o arquivo é lido em blocos de 16 MB e relido a cada rodada com `rewind()`, mantendo em RAM apenas as estruturas de tamanho $O(V)$.

A regra de desempate, deixada abstrata na especificação, foi fixada como peso crescente e, havendo empate, menor origem e depois menor destino, o que torna a escolha de arestas determinística. Por fim, além da verificação de mesma componente feita na varredura, há uma segunda verificação na fase de união, necessária porque as fusões da própria rodada alteram os componentes. Essa segunda checagem evita duplicar arestas, quando duas componentes se escolhem mutuamente, e evita fechar ciclos, em fusões encadeadas.

O peso total da árvore também é obtido por acumulação parcial dentro de cada componente ao longo da construção. Cada componente guarda a soma parcial da árvore já formada dentro dele. Quando dois componentes se fundem, o peso do componente resultante é a soma das duas parciais mais o peso da aresta que os uniu, gravado na raiz que sobrevive à união. Ao final resta um único componente e o peso total da MST já está na raiz. As duas versões usam essa acumulação; na paralela, apenas o processo 0 mantém as somas parciais, pois é ele quem registra as arestas escolhidas.

Na versão paralela, o union-find é replicado em todos os processos, sincronizado por um `MPI_Allreduce` por rodada, em vez de uma estrutura distribuída, o que dispensa comunicação no passo de união ao custo de manter o vetor de componentes em cada processo. Completam essas escolhas dois detalhes do vetor de menores. Ele cresce dobrando a capacidade a cada expansão, ideia que reduz o número de chamadas a `realloc`, e cada expansão inicializa apenas as novas posições ao fim do vetor com o sentinela `DBL_MAX`, sem alterar as entradas já existentes no início. Esse mesmo sentinela marca “Nenhuma” nos componentes sem aresta de saída.

4 Comunicação entre as máquinas

A leitura e a distribuição dos dados seguem o modelo de que apenas o processo 0 abre e lê o arquivo, em blocos de 16 MB, e repassa as arestas aos demais por troca de mensagens ponto-a-ponto.

A atribuição dos blocos aos processos é *round-robin*: o bloco de índice j é atribuído ao processo $(j \bmod p)$, onde p é o número de processos. A partir do número total de arestas, difundido a todos por `MPI_Bcast`, e do tamanho de bloco, cada processo calcula localmente quantas arestas formarão a sua fatia, de modo que todos conhecem o tamanho da própria fatia sem comunicação adicional e alocam o buffer exato antes de receber.

A transferência usa comunicação não bloqueante com duplo buffer, para sobrepor a leitura do disco ao envio pela rede. No processo 0, dois buffers se alternam: enquanto um bloco é enviado, o próximo já é lido do disco para o outro buffer, e um `MPI_Wait` garante que um buffer só seja reutilizado depois que o envio anterior conclui. Nos processos trabalhadores, cada um sabe de antemão quais blocos lhe cabem e o tamanho de cada um, pois a atribuição é round-robin e o tamanho de qualquer bloco é deduzido localmente a partir do número total de arestas e do tamanho de bloco. Por isso, antes de terminar de processar o bloco atual, o processo já calcula o tamanho do próximo e posta um `MPI_Irecv` para recebê-lo, adiantando a recepção enquanto o bloco atual é processado. Assim, a latência de disco e de rede fica escondida atrás da computação.

Para enviar as structs, são criados dois tipos derivados com `MPI_Type_contiguous`, cada um tratando a struct inteira como um bloco contíguo de bytes (`MPI_BYTE`): um para as arestas distribuídas e outro para o vetor de menores usado na redução. Isso funciona porque todas as máquinas do cluster são idênticas e têm o mesmo layout de memória para as structs, o que dispensa descrever campo a campo os deslocamentos.

As operações coletivas concentram a sincronização. Um `MPI_Allreduce` com `MPI_MAX` sobre o maior identificador local determina o número de vértices. A cada rodada, um `MPI_Allreduce` com o operador comutativo `op_menor_aresta()` combina os `menor_local[]` em `menor_global[]`.

5 Características da máquina de execução

As versões sequencial e paralela foram executadas em máquinas do cluster do laboratório, todas de configuração idêntica. A Tabela 3 resume essa configuração.

Item	Especificação
Processador (clock)	AMD Ryzen 5 PRO 5650G, clock de 0,40 a 4,47 GHz
Núcleos / threads	6 núcleos físicos, 12 threads (2 por núcleo)
Memória RAM	7,1 GiB
Sistema operacional	Ubuntu 24.04.2 LTS (kernel 6.17.0-23-generic)

Tabela 3: Configuração da máquina utilizada nas execuções.

6 Testes e análise

A correção foi verificada comparando a saída paralela com a sequencial: em todas as execuções o peso total da MST foi idêntico, 75117,848733921419, o que confirma que a versão paralela gera a mesma árvore que a versão sequencial.

O desempenho foi medido tomando como referência o tempo da versão sequencial, 384,49 s. As Figuras 1 a 5 reúnem os resultados para 4, 8, 12 e 16 processos.

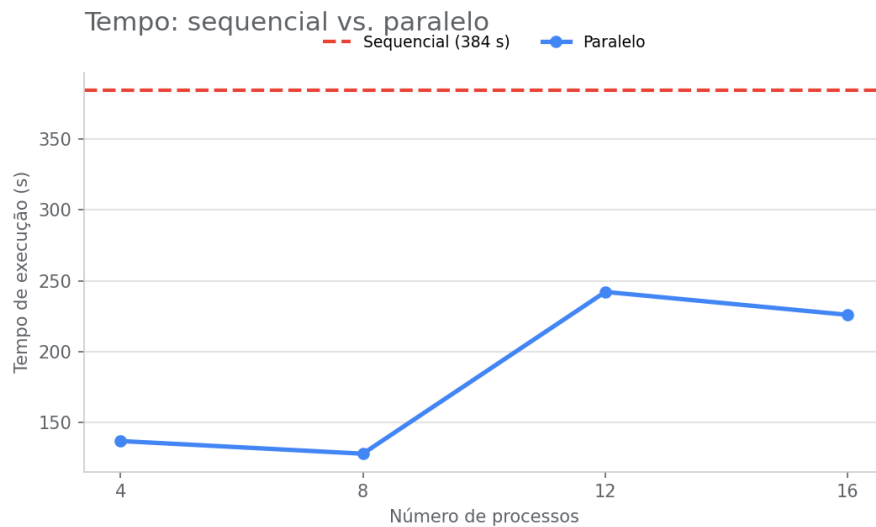


Figura 1: Tempo: sequencial vs. paralelo.

Fonte: elaborado pelo autor (2026).

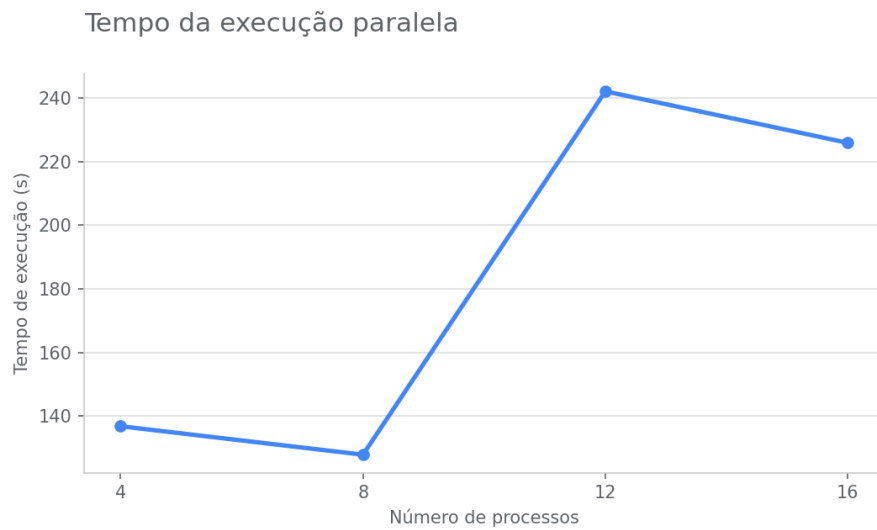


Figura 2: Tempo da execução paralela.
Fonte: elaborado pelo autor (2026).

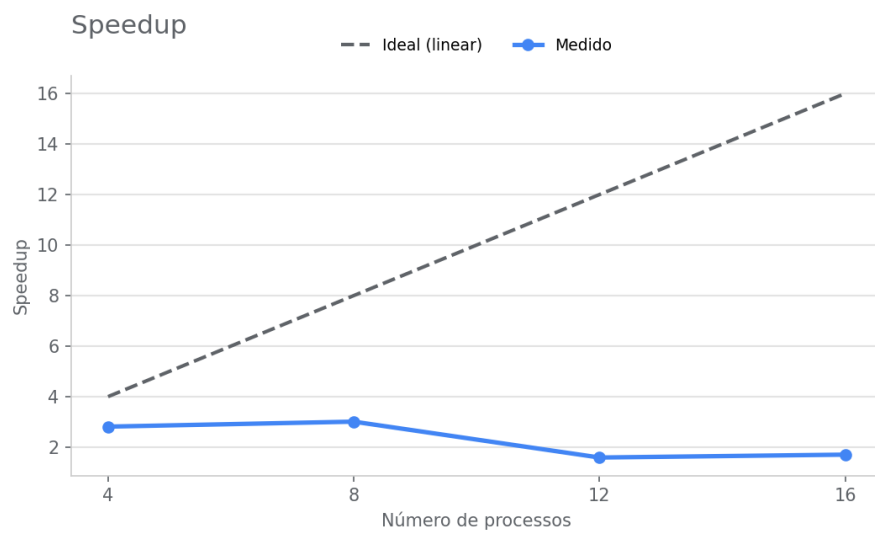


Figura 3: Speedup.
Fonte: elaborado pelo autor (2026).

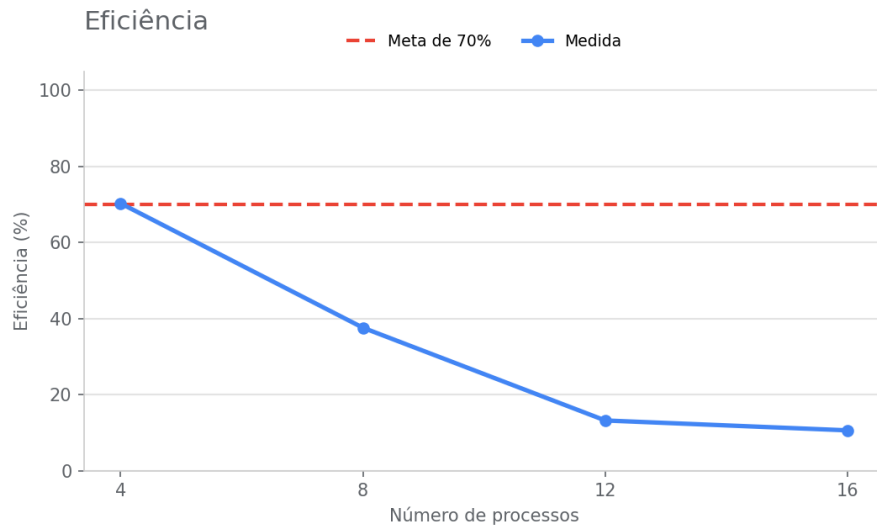


Figura 4: Eficiência.
Fonte: elaborado pelo autor (2026).

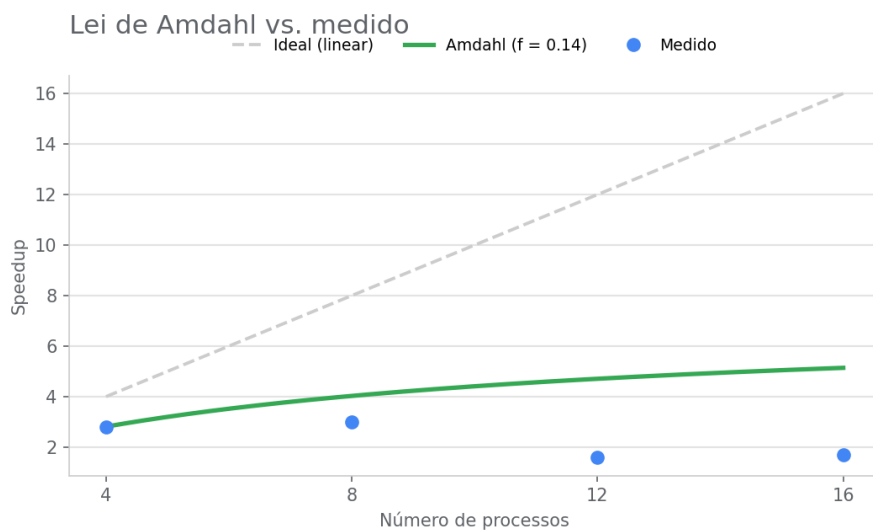


Figura 5: Speedup vs. Lei de Amdahl.
Fonte: elaborado pelo autor (2026).

A configuração de 4 processos atingiu eficiência de 70,29%, atendendo ao requisito de eficiência mínima de 70% exigido no enunciado. O menor tempo absoluto ocorreu com 8 processos, 127,81 s, contra os 384,49 s da versão sequencial. A partir de 8 processos, porém, o tempo volta a crescer (12 processos: 242,10 s; 16 processos: 225,89 s) e a eficiência cai de forma contínua.

7 Evidências de execução

Esta seção reúne as capturas de tela que evidenciam o funcionamento correto dos programas. A Figura 6 mostra a compilação e a execução da versão sequencial. As Figuras 7 a 10 mostram a versão paralela, com os resultados para 4, 8, 12 e 16 processos.

```
rgm47539@l1m19u24:/home/local/rgm47539/mpi$ make run-seq
./IgorRoberto_47539_sequencial graph.bin
Ok..

Distancia da arvore geradora minima: 75117.848733921419
Tempo de execucao: 384.491200 segundos
rgm47539@l1m19u24:/home/local/rgm47539/mpi$
```

Figura 6: Execução sequencial.
Fonte: elaborado pelo autor (2026).

```
rgm47539@l1m27u24:/home/local/rgm47539/mpi$ make run
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified
Authorization required, but no authorization protocol specified

[rank 0 em l1m27u24] fatia = 200278016 arestas
[rank 2 em l1m29u24] fatia = 200214528 arestas
[rank 1 em l1m28u24] fatia = 200278016 arestas
[rank 3 em l1m30u24] fatia = 199229440 arestas

Resultado: peso = 75117.848733921419
Eficiencia= 70.29%
Tempo= 136.753323 s
rgm47539@l1m27u24:/home/local/rgm47539/mpi$
```

Figura 7: Execução com 4 processos.
Fonte: elaborado pelo autor (2026).


```

rgm47539@l1m19u24:/home/local/rgm47539/mpi$ make run

LAM 7.1.4/MPI 2 C++/ROMIO - Indiana University

n-1<5612> ssi:boot:base:linear: booting n0 (172.26.1.157)
n-1<5612> ssi:boot:base:linear: booting n1 (l1m20u24)
n-1<5612> ssi:boot:base:linear: booting n2 (l1m27u24)
n-1<5612> ssi:boot:base:linear: booting n3 (l1m22u24)
n-1<5612> ssi:boot:base:linear: booting n4 (l1m11u24)
n-1<5612> ssi:boot:base:linear: booting n5 (l1m12u24)
n-1<5612> ssi:boot:base:linear: booting n6 (l1m13u24)
n-1<5612> ssi:boot:base:linear: booting n7 (l1m14u24)
n-1<5612> ssi:boot:base:linear: booting n8 (l1m28u24)
n-1<5612> ssi:boot:base:linear: finished
[rank 0 em l1m19u24] fatia = 100663296 arestas
[rank 2 em l1m27u24] fatia = 100599808 arestas
[rank 6 em l1m13u24] fatia = 99614720 arestas
[rank 3 em l1m22u24] fatia = 99614720 arestas
[rank 5 em l1m12u24] fatia = 99614720 arestas
[rank 7 em l1m14u24] fatia = 99614720 arestas
[rank 1 em l1m20u24] fatia = 100663296 arestas
[rank 4 em l1m11u24] fatia = 99614720 arestas

Resultado: peso = 75117.848733921419
Eficiencia= 37.60%
Tempo= 127.807154 s
rgm47539@l1m19u24:/home/local/rgm47539/mpi$

```

Figura 8: Execução com 8 processos.

Fonte: elaborado pelo autor (2026).

```

rgm47539@l1m11u24:/home/local/rgm47539/mpi$ make run
[rank 0 em l1m11u24] fatia = 67108864 arestas
[rank 7 em l1m22u24] fatia = 66060288 arestas
[rank 6 em l1m21u24] fatia = 67045376 arestas
[rank 1 em l1m12u24] fatia = 67108864 arestas
[rank 3 em l1m14u24] fatia = 67108864 arestas
[rank 11 em l1m31u24] fatia = 66060288 arestas
[rank 9 em l1m28u24] fatia = 66060288 arestas
[rank 2 em l1m13u24] fatia = 67108864 arestas
[rank 10 em l1m29u24] fatia = 66060288 arestas
[rank 8 em l1m27u24] fatia = 66060288 arestas
[rank 5 em l1m20u24] fatia = 67108864 arestas
[rank 4 em l1m19u24] fatia = 67108864 arestas

Resultado: peso = 75117.848733921419
Eficiencia= 13.23%
Tempo= 242.101432 s

```

Figura 9: Execução com 12 processos.

Fonte: elaborado pelo autor (2026).

```

rgm47539@l1m03u24:/home/local/rgm47539/mpi$ nano Makefile
rgm47539@l1m03u24:/home/local/rgm47539/mpi$ make run
[rank 0 em l1m03u24] fatia = 50331648 arestas
[rank 11 em l1m29u24] fatia = 49283072 arestas
[rank 10 em l1m28u24] fatia = 50268160 arestas
[rank 2 em l1m12u24] fatia = 50331648 arestas
[rank 8 em l1m22u24] fatia = 50331648 arestas
[rank 6 em l1m20u24] fatia = 50331648 arestas
[rank 14 em l1m05u24] fatia = 49283072 arestas
[rank 12 em l1m31u24] fatia = 49283072 arestas
[rank 3 em l1m13u24] fatia = 50331648 arestas
[rank 9 em l1m27u24] fatia = 50331648 arestas
[rank 15 em l1m15u24] fatia = 49283072 arestas
[rank 7 em l1m21u24] fatia = 50331648 arestas
[rank 5 em l1m19u24] fatia = 50331648 arestas
[rank 4 em l1m14u24] fatia = 50331648 arestas
[rank 1 em l1m11u24] fatia = 50331648 arestas
[rank 13 em l1m23u24] fatia = 49283072 arestas

Resultado: peso = 75117.848733921419
Eficiencia= 10.63%
Tempo= 225.892421 s

```

Figura 10: Execução com 16 processos.

Fonte: elaborado pelo autor (2026).

8 Conclusão

Este trabalho implementou a construção de uma Árvore Geradora Mínima em memória distribuída com C e MPI, em uma versão sequencial e uma paralela executada em 4, 8, 12 e 16 processos.

A correção foi confirmada pelo peso idêntico da MST em todas as configurações.

Quanto ao desempenho, a versão paralela superou a sequencial em todas as configurações testadas, e a configuração de 4 processos atendeu ao requisito de eficiência mínima de 70%, com 70,29%. O menor tempo foi obtido com 8 processos. A partir desse ponto, acrescentar processos deixou de compensar: o tempo voltou a subir e a eficiência caiu, porque a leitura e a distribuição serial do arquivo pelo processo 0 e a sincronização global a cada rodada passam a dominar o tempo total à medida que mais máquinas participam.

O material deste trabalho foi entregue reunido em um único diretório, composto pelos seguintes arquivos:

- **paralelo**: código-fonte da versão paralela;
- **sequencial**: código-fonte da versão sequencial;
- **Makefile**: script para compilação dos programas;
- **readme.txt**: nome do autor e comandos de execução;
- **Logs/**: pasta com o log de execução para debug;
- este relatório, em formato PDF.

Referências

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*. 3. ed. MIT Press, 2009.
- [2] P. Feofiloff. *A estrutura union-find*. Análise de Algoritmos, IME-USP. Disponível em: https://www.ime.usp.br/~pf/analise_de_algoritmos/aulas/union-find.html. Acesso em: 2026.
- [3] J. JáJá. *An Introduction to Parallel Algorithms*. Addison-Wesley, 1992.
- [4] UEMS – Universidade Estadual de Mato Grosso do Sul. *Programação Paralela e Distribuída*. Página da disciplina, 2026. Disponível em: <https://www.comp.uems.br/~rubens/ppd.html>.